

Generating Compilers for Qubit Mapping and Routing

ABTIN MOLAVI, University of Wisconsin-Madison, USA

AMANDA XU, University of Wisconsin-Madison, USA

ETHAN CECCHETTI, University of Wisconsin-Madison, USA

SWAMIT TANNU, University of Wisconsin-Madison, USA

AWS ALBARGHOUTHI, University of Wisconsin-Madison, USA

To evaluate a quantum circuit on a quantum processor, one must find a mapping from circuit qubits to processor qubits and plan the instruction execution while satisfying the processor's constraints. This is known as the *qubit mapping and routing* (QMR) problem. High-quality QMR solutions are key to maximizing the utility of scarce quantum resources and minimizing the probability of logical errors affecting computation. The challenge is that the landscape of quantum processors is incredibly diverse and fast-evolving. Given this diversity, dozens of papers have addressed the QMR problem for different qubit hardware, connectivity constraints, and quantum error correction schemes by a developing a new algorithm for a particular context. We present an alternative approach: automatically generating qubit mapping and routing compilers for arbitrary quantum processors. Though each QMR problem is different, we identify a common core structure—*device state machine*—that we use to formulate an *abstract QMR problem*. Our formulation naturally leads to a compact domain-specific language for specifying QMR problems and a powerful parametric algorithm that can be instantiated for any QMR specification. Our thorough evaluation on case studies of important QMR problems shows that generated compilers are competitive with handwritten, specialized compilers in terms of runtime and solution quality.

CCS Concepts: • **Hardware** → **Quantum computation**; • **Software and its engineering** → **Architecture description languages**; **Compilers**.

Additional Key Words and Phrases: qubit layout, simulated annealing

ACM Reference Format:

Abtin Molavi, Amanda Xu, Ethan Cecchetti, Swamit Tannu, and Aws Albarghouthi. 2026. Generating Compilers for Qubit Mapping and Routing. *Proc. ACM Program. Lang.* 10, POPL, Article 78 (January 2026), 30 pages. <https://doi.org/10.1145/3776720>

1 Introduction

Quantum computation promises to surpass classical methods in important domains, potentially unlocking breakthroughs in materials science, chemistry, machine learning, and beyond. Quantum computing is at an inflection point: scientists are scaling quantum hardware [12, 21], demonstrating practical quantum error correction protocols [16], and exploring promising application domains [30]. However, to fully realize the potential of quantum hardware available today and on the horizon, we need optimizing quantum circuit compilers. A compiler must convert architecture-independent, circuit-level descriptions of quantum programs to a form executable on a target *quantum processing unit* (QPU). Inefficient compilation that induces significant runtime overhead is unacceptable. For one, access to quantum compute is limited and costly. Further, quantum computation is error-prone,

Authors' Contact Information: [Abtin Molavi](#), University of Wisconsin-Madison, USA, amolavi@wisc.edu; [Amanda Xu](#), University of Wisconsin-Madison, USA, axu44@wisc.edu; [Ethan Cecchetti](#), University of Wisconsin-Madison, USA, cecchetti@wisc.edu; [Swamit Tannu](#), University of Wisconsin-Madison, USA, stannu@wisc.edu; [Aws Albarghouthi](#), University of Wisconsin-Madison, USA, aws@cs.wisc.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/1-ART78

<https://doi.org/10.1145/3776720>

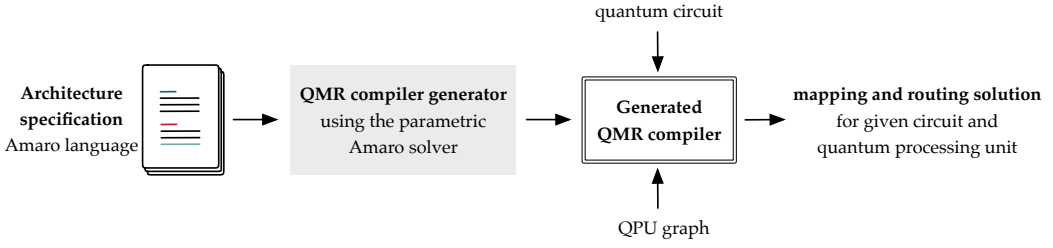


Fig. 1. Overview of our approach to QMR compiler generation. Given a specification of a QMR problem in the Amaro language, we generate a compiler by instantiating the parametric Amaro solver. The generated compiler takes a quantum circuit and target QPU graph as input and produces a mapping and routing solution.

and longer computations are associated with a higher probability of a logical error, even when quantum error-correcting codes are applied.

To enable execution of a quantum circuit on a target QPU, a compiler must find a *mapping* from circuit qubits to physical locations on the QPU and plan the *routing* of quantum instructions (gates) in a way that is compliant with the QPU’s physical and logical constraints. This is known as the *qubit mapping and routing problem* (QMR).

The challenge for compiler designers is that the landscape of target quantum architectures is incredibly diverse and fast-evolving. First, there are several competing hardware realizations of an individual physical qubit, such as superconducting circuits, neutral atoms, trapped ions, and photons. Each physical realization of qubits imposes its own unique constraints on QMR. For example, superconducting qubits are fixed in place, while neutral atom qubits can be shuttled in physical space. Second, qubits can be arranged and connected in a variety of ways. For example, superconducting circuits can be arranged in a linear array or a grid. Third, going up the abstraction ladder, *quantum-error correction* (QEC) schemes encode a logical qubit using several physical qubits, and each QEC scheme imposes its unique architectural constraints.

The diversity in quantum architectures yields an array of QMR problems to tackle for different qubit hardware, connectivity constraints, and quantum error correction schemes. A recent survey [67] explicitly enumerates dozens of papers which address a variant of the QMR problem as their primary focus. Table 1 shows a selection of QMR problems studied in the literature, highlighting the diversity of the considered constraints. For each new set of mapping and routing constraints, researchers establish hardness results, identify connections to graph-theoretic problems, and develop specialized compilation algorithms. Ideally, we would prefer to avoid restarting this process from scratch for each new emerging architecture.

Therefore, in this work we ask the following question:

Can we automatically synthesize a compiler from a specification of architectural constraints?

To this end, we construct a framework called Amaro (Abstract Mapping and ROuting) that unifies and generalizes QMR problems, illustrated in Fig. 1. Though each QMR problem is different, we identify a common core structure that we use to define an *abstract QMR problem*. Then, each architecture-specific problem is a different concrete instantiation of the abstract problem. Our abstract QMR problem is based on the view of a mapping and routing solution as a sequence of *device states*. A device state captures the current qubit mapping and which gates are evaluated in parallel at a given execution time step. The architectural constraints dictate which states and transitions between states are valid.

Our formulation of the abstract QMR problem naturally leads to the Amaro language, a domain-specific language for specifying QMR problems. A program in the Amaro language defines a *device*

Table 1. A selection of QMR problems that we use as case studies

Case study	Amaro LoC	Prior work
Near-term superconducting (NISQMR)	12	[9, 35, 39, 52, 62]
Near-term superconducting with variable error (NISQ-VE)	30	[41, 59]
Trapped-ion compilation (TIQMR)	51	[3, 42]
Reconfigurable atom array compilation (RAA)	116	[56, 57, 61]
Surface code mapping and routing (SCMR)	40	[20, 40]
Multi-qubit lattice surgery scheduling (MQLSS)	56	[54]
Interleaved logical qubit compilation (ILQ)	44	[60]

state machine that describes the QPU’s physical and logical constraints. With a few lines of code, we can specify a new QMR problem and automatically generate a compiler for it. For example, the definition of the most well-studied QMR problem, for *noisy intermediate-scale quantum* (NISQ) architectures, is just 12 lines in the Amaro language (Table 1 shows a selection of QMR problems that we use as case studies along with line counts).

We demonstrate that our abstract QMR problem can be solved with a simple parametric solver that can be instantiated for any QMR problem. Given a specification of a QMR problem in the Amaro language, we automatically generate a compiler for it by instantiating the parametric Amaro solver. Generally, QMR problems have been shown to be NP-hard [40, 55] and constraint-based solutions have been shown to be unscalable. Our solver, instead, is approximate in nature. It constructs the mapping and routing solution by incrementally building a sequence of device states, attempting to maximize the number of gates that can be executed in parallel. We call this approach *maximal-state construction* and study the properties of QPUs for which our solver can find maximal states. We identify two desirable properties, *monotonicity* and *non-interference*. It turns out, for all practical QPUs, we can find maximal states (via monotonicity), and for some QPUs (like in the NISQ setting), we can find *maximum* states without requiring search (via non-interference)—speeding up the algorithm. For managing the combinatorial explosion, we use the classic and simple *simulated annealing* algorithm [31]. Our solver is suitable for industrial-scale quantum circuit compilation because it is simple, highly configurable, and amenable to parallelization.

We evaluate our approach with several case studies of important QMR problems considered in prior work (see Table 1), including very recently introduced QMR problems for fault-tolerant quantum computers [54, 60]. Qualitatively, our results demonstrate the generality and versatility of our abstract QMR formulation: we are able to concisely specify QMR problems for noisy and fault-tolerant quantum architectures on a variety of hardware realizations. Quantitatively, we perform an experimental evaluation on a comprehensive circuit benchmark suite to assess the performance of our solver. Our results indicate that our generated compilers are competitive with handwritten compilers in terms of runtime and solution quality (details in Sec. 7). For example, our solver finds solutions with the same cost or better than the leading industrial toolkit, QISKIT [26], for half of our benchmarks. On some QMR domains, we even outperform the prior state-of-the-art. For the case of interleaved logical qubits, our solutions are strictly higher-quality than the baseline for 93% of cases. We envision that our approach will simplify development of quantum compilers for the many new and emerging quantum architectures.

To summarize, our contributions are the following:

- An abstract formulation of the mapping and routing problem, based on device state machines, that presents a uniform way of thinking of the zoo of QMR problems (Sec. 3).

- A specification language for QMR problems that enables concise expression of the unique constraints of a particular architecture family. (Sec. 4)
- A powerful parametric solver that can be automatically instantiated into a compiler for a given QMR problem from a specification (Sec. 5)
- An extensive empirical evaluation demonstrating the generality of our approach and the quality of the synthesized compilers in comparison to handwritten compilers. (Secs. 6 and 7)

2 Background and Overview of our Approach

In this section, we begin by giving an overview of quantum circuits, providing the relevant background for this work to readers unfamiliar with quantum computing. We then introduce the QMR problem through two key examples of target QPUs. By studying these two examples, we highlight the commonalities and motivate the abstract QMR problem and our specification language.

2.1 Quantum Circuits

We give a brief overview of quantum circuits. Since we are interested in the mapping and routing problem, it suffices to consider the structure of quantum circuits and not their full semantics. A comprehensive introduction can be found in any standard quantum computing text (e.g., [44]).

Qubits. The unit of data in quantum computing is called the *qubit*. A qubit can be one of the two *computational basis* states, 0 and 1, or a linear combination of the two, with complex-number coefficients called *amplitudes*. The state of n qubits is described by 2^n amplitudes.

Gates. Quantum states are transformed by operations called *quantum gates*. In this work, we focus on single-qubit and two-qubit gates. One important single-qubit gate is the T gate. The T gate leaves the 0 state unchanged, and applies a *phase-shift* to the 1 state, multiplying its amplitude by $e^{i\pi/4}$. The T gate plays an important role in fault-tolerant quantum computation. While necessary for universality—the ability to approximate any quantum computation to arbitrary precision— T gates are typically expensive to implement in the context of quantum error correction.

A common two-qubit gate is the CX gate. The CX gate is named for its action on the computational basis states as a “conditional-not.” If the first argument, called the *control*, is 1, the CX applies a NOT to the second argument, which is called the *target*. If the control is in the 0 state, the gate has no effect. Another important two-qubit gate is the $SWAP$ gate, which swaps the values of two qubits.

Circuits. Quantum gates can be composed to produce a quantum circuit. Fig. 2 shows a simple quantum circuit. This example merely demonstrates the structure of circuits; the computation it performs is not significant. Each horizontal wire represents a qubit—two qubits are present in this example—and the circuit is read from left to right. In this circuit, we first apply a CX gate to the two qubits (the top qubit is the control and the bottom qubit is the target), then a T gate to each qubit. Equivalently, this circuit can be written as a sequence of instructions: $CX\ q_1\ q_2; T\ q_1; T\ q_2$.

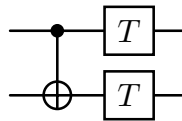


Fig. 2. A simple quantum circuit

2.2 An Introduction to QMR Problems

In this section, we introduce qubit mapping and routing through two key examples of target QPUs. We chose two ostensibly very different QMR problems, one targeting noisy quantum computers and the other targeting fault-tolerant quantum computers. In comparing these two problems, we highlight a shared core structure with problem-specific parametric components.

Both QMR problems reduce to finding an execution plan for a given circuit on a given QPU. We can express an execution plan as a sequence of *device states* of the QPU. Intuitively, each state in the sequence represents a time step of execution. The differences between QMR problems emerge as constraints that state sequences must satisfy to be considered valid solutions and *cost* functions defining solution optimality.

We describe our two example QMR problems with an eye towards this unifying perspective, summarizing each problem in terms of the constraints imposed on the state sequences.

2.2.1 Noisy-Intermediate Scale Quantum Computers. First, we consider mapping and routing for noisy-intermediate scale quantum (NISQ) architectures. These QPUs consist of up to hundreds of physical qubits and do not implement error-correction. We will call this the NISQMR problem. NISQMR is well-studied with a variety of proposed solutions appearing in the literature, ranging from greedy heuristic maximization [9, 35] to A^* search [68] to reductions to satisfiability [36, 39, 66].

Mapping and Routing for NISQ Devices. Because of spatial limitations, many NISQ QPUs, like the devices developed by IBM [22] and Google [13, 15], only support two-qubit gates between certain pairs of qubits. We can represent a particular QPU with a *connectivity graph*. The connectivity graph includes an edge between a pair of qubits if and only if the QPU supports a two-qubit gate between them. A simple connectivity graph is shown in Fig. 3a (a linear graph where adjacent physical qubits are connected by an edge). The goal of the compiler is to find a *qubit map* from the qubits which appear in the circuit to the physical qubits of the QPU such that two-qubit gates are executable, which is to say that the circuit qubits the gate acts on are mapped to adjacent qubits. For example, suppose we wish to execute the circuit in Fig. 3b on this QPU. The first gate, g_0 , is between qubits q_0 and q_1 , so we choose a map that maps these to a pair of adjacent physical qubits, such as p_0 and p_1 . Likewise, to execute the second gate, g_1 , we map q_2 and q_3 to p_2 and p_3 . However, we cannot execute either of the remaining gates, g_2 and g_3 , with this qubit map because there is no edge between the qubits the gates act on.

Changing the Qubit Map via SWAP Gates. There is often no single static map such that all two-qubit gates are executable. Instead, the map is transformed over the course of execution by the insertion of SWAP gates. SWAP gates exchange the states of two adjacent qubits. In our example, to execute the CX gates g_2 and g_3 (CX q_1 q_3 and CX q_0 q_2), we insert a SWAP operation SWAP p_1 p_2 .

A representation of our full mapping and routing solution is depicted in Fig. 3c. Each box represents the *device state* of the QPU at a particular time step; each state is an assignment of some subset of the gates of the circuit to an edge in the connectivity graph and has an associated qubit map. For example, in state 1, gate g_0 is assigned to the edge (p_0, p_1) while gate g_1 is assigned to the edge (p_2, p_3) . The SWAP operations inserted between states dictate how the qubit map changes.

Each additional noisy gate increases the probability of error in the execution of the circuit, and two-qubit gates like the SWAP gate are especially costly, with an error rate that is typically an order of magnitude higher than single qubit gates. Therefore, the goal is to find a solution which minimizes the number of inserted SWAP gates.

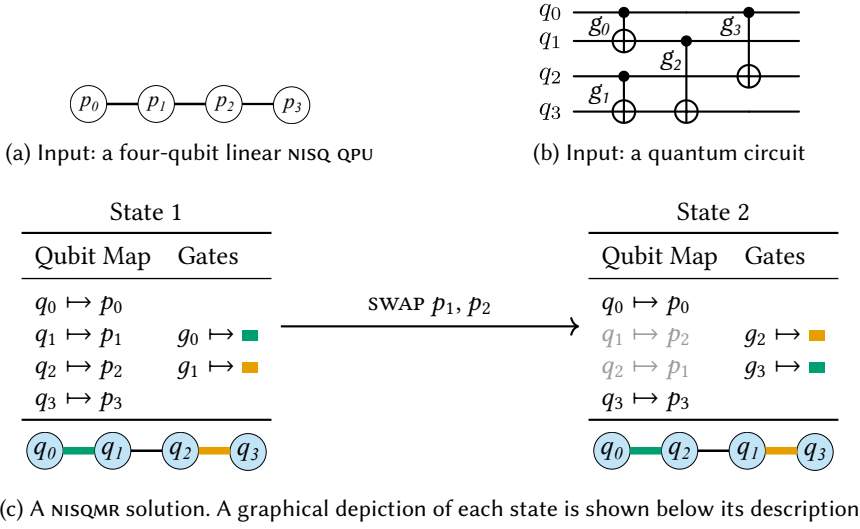


Fig. 3. Overview of the NISQMR problem

In summary, the NISQMR problem consists of the following components.

- **Input:** a circuit and a NISQ QPU (represented as a connectivity graph).
- **Output:** an execution plan for the circuit on the QPU as a sequence of states. Each state consists of a map from circuit qubits to QPU physical qubits and a set of two-qubit gates that are executed.
- **Gate Realization:** the plan associates each gate with a *realization*, the edge along which it is implemented.
- **Transitions:** between states, the qubit map can be transformed by SWAP gates, which define the valid *transitions*.
- **Cost:** the goal is to minimize the number of added SWAP operations.

2.2.2 Surface Code. Now we turn to processors implementing the *surface code* [11], a leading approach for quantum error correction that has recently been demonstrated in hardware [5, 14, 16]. We refer to the QMR problem for surface code processors as SCMR. In the surface code, a two-dimensional array of physical qubits is used to encode a fault-tolerant *logical qubit*. A surface code logical qubit is shown in the inset on the left of Fig. 4a. The large circles denote physical qubits which carry the logical state, while the small circles denote physical qubits which are repeatedly measured to detect errors. A surface code QPU consists of several surface code logical qubits embedded in the same lattice of physical qubits. We can represent a QPU with a grid graph where each vertex represents a logical qubit, and we include an edge between adjacent logical qubits, not including diagonals, as shown on the right of Fig. 4a.

Two-Qubit Gates as Paths. Two-qubit gates between surface code logical qubits are implemented via a procedure called *lattice surgery* [19]. To apply a lattice surgery CX gate, we need to find a path of logical qubits on the grid from the control qubit to the target qubit (through *ancilla qubits*). The path must connect a horizontal boundary of the control (the top or bottom edge) to a vertical boundary of the target, making at least one “bend”. The paths for gates which are executed simultaneously cannot cross. The challenge of SCMR is thus to map circuit qubits to QPU logical qubits and plan paths such that we avoid conflicts where one gate blocks another.

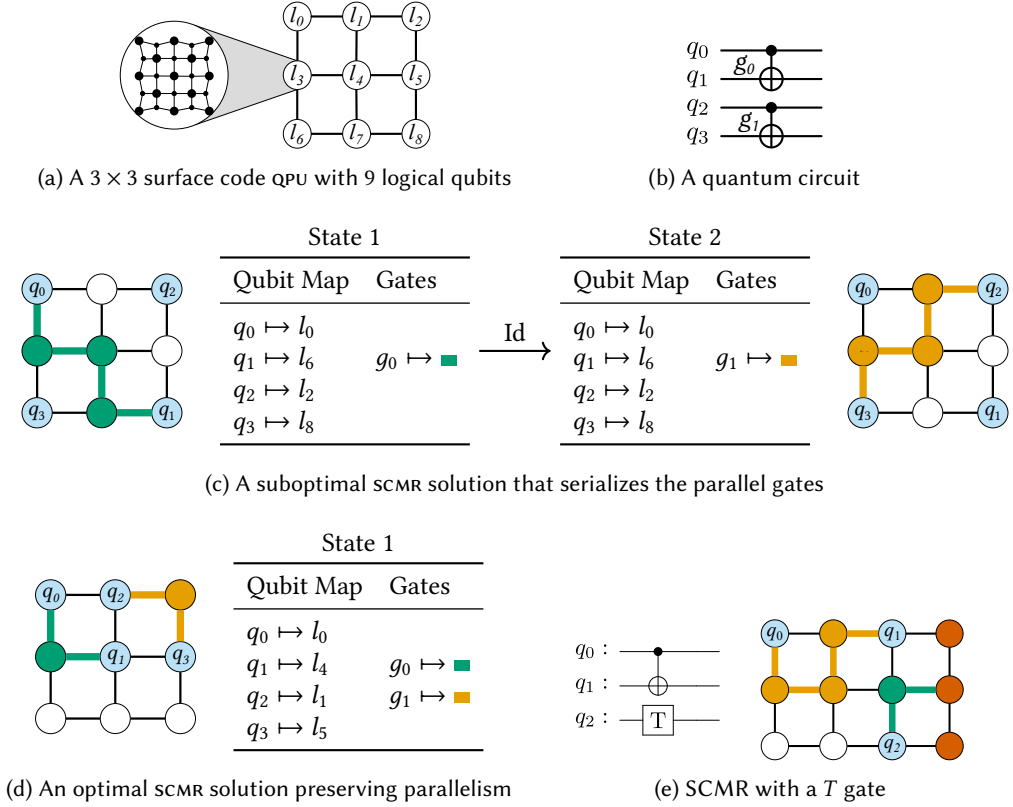


Fig. 4. Overview of the SCMR problem (examples adapted from Molavi et al. [40])

For example, say we wish to execute the circuit in Fig. 4b on our 3×3 QPU in Fig. 4a. Theoretically, the two cx gates of the circuit can be executed in parallel. However, we need to choose the qubit map carefully to enable parallel execution. Consider Fig. 4c. With this qubit map, there is no way to simultaneously execute the two gates, resulting in a two-time-step solution because any paths from q_0 to q_1 and q_2 to q_3 must cross. Note that, in this setting, the qubit map does not change between states. On the other hand, Fig. 4d shows how a different qubit map yields a single-state solution. Solutions with fewer states are preferable because they save quantum compute resources at run time, and have a lower probability of logical error, which accumulates with each state.

Routing T Gates. The surface code problem must also account for T gates. While the T gate is a single-qubit gate, it cannot be applied directly to surface code logical qubits. The main proposal for addressing this limitation is called magic state injection [6]. In this protocol, a T gate is implemented via a lattice surgery cx gate between the input to the T gate and another logical qubit prepared in a so-called “magic state.” Magic state qubits are stored in designated locations on the QPU, which we represent with special distinguished vertices in our grid graph representation.

We provide an example in Fig. 4e. Since this circuit contains a T gate, we need to define magic state qubit locations. The 3×3 architecture has been extended with a column of magic state qubits along the right side, indicated with orange vertices. An optimal mapping and routing solution with one state is shown. We have two simultaneous connections: one between qubits q_0 and q_1 corresponding to the cx; the other between q_2 and a magic state qubit, corresponding to the T gate.

In summary, the SCMR problem consists of the following components.

- **Input:** a circuit and a surface code QPU (represented as a grid graph).
- **Output:** an execution plan for the circuit on the QPU as a sequence of states. Each state consists of a map from circuit qubits to QPU logical qubits and a set of two-qubit gates and T gates that are executed.
- **Gate Realization:** the plan associates each gate with a *realization*, the path along which it is implemented.
- **Transitions:** between steps, the qubit map remains constant; that is, the only valid *transition* is the identity transformation.
- **Cost:** the goal is to minimize the number of time steps.

2.2.3 Abstracting the QMR Problem. As these two example problems illustrate, *each QMR problem can be seen as a specialization of the same abstract QMR problem*. Specifically, a QMR problem specifies a *device state machine* with constraints on the states and transitions between them. This observation has two major consequences. First, we can specify a particular QMR problem succinctly by defining only the unique components of the device state machine:

- the problem-specific constraints on valid states (which gates are realizable and how they are realized),
- the transition relation which describes how the qubit map can change between states, including any additional data that the QPU graph carries, and
- the optimization objective defining the cost of a solution.

The Amaro language is designed to facilitate such a description. Second, we can define a generic solver which is parameterized by these components and tries to find an optimal sequence of device states that solves the QMR problem. We use the name Amaro to refer to either of these components when the meaning is clear from context.

2.3 The Amaro Language for Specifying QMR Problems

To enable painless specification of new QMR problems, we design the Amaro language, a domain-specific language tailored to our abstract QMR problem. In this subsection, we introduce the key features of the Amaro language through an example, with a detailed language definition to follow in Sec. 4. An Amaro program corresponds to a concrete QMR problem, like NISQMR or SCMR. Through a sequence of data and function definitions written in a simple functional expression language, an Amaro program defines a function from a circuit and a QPU (defined as a graph) to a *set* of mapping and routing solutions (as sequences of states), each associated with a cost. The Amaro solver will have to find one of the sequences that minimizes the cost.

For example, the Amaro program for the NISQMR problem is shown in Fig. 5. Notice that since Amaro is purpose-built for specifying QMR problems, the programs are concise. Here, we fully specify the NISQMR problem in 12 lines.

Program Structure. An Amaro program takes as input a circuit, referred to implicitly using its constituent gates (`Inst` variable), and a QPU represented as a graph of device qubits (`Arch` variable). The program is divided into a “RouteInfo” block (delimited by `RouteInfo:`) and a “TransitionInfo” block (delimited by `TransitionInfo:`). Together, these two blocks define the device state machine:

- The RouteInfo block specifies the constraints on a valid QPU state in terms of how a gate is associated with its physical realization.
- The TransitionInfo block specifies the constraints on a valid transition between QPU states.

RouteInfo Block. For NISQMR, a QPU state associates a CX gate with a pair of QPU locations. The first two lines of the RouteInfo block for this problem encode this information.

The first line defines the `GateRealization` datatype. In this case, the `GateRealization` datatype has a field which is a pair of elements of type `Loc`. The built-in `Loc` type represents locations on a QPU. Amaro has a built-in notion of an abstract QPU as a graph over vertices of type `Loc`. The second line specifies that CX gates are the gate type to route.

```
GateRealization{edge : (Loc, Loc)}
routed_gates = [CX]
```

The rest of the block defines a function `realize_gate` which says that a `Loc` pair is a valid realization for a gate precisely when the pair is an edge between the gate's qubits.

```
// realize_gate : (Arch, State, Gate) → List[GateRealization]
realize_gate = map(|x| → GateRealization{edge = x},
  Arch.edges_between(State.map[Gate.qubits[0]], State.map[Gate.qubits[1]]))
```

The `realize_gate` function is included in every Amaro file. It has three implicit parameters, a QPU `Arch`, a state `State`, and a gate `Instr`. It returns all possible implementations of the gate which can be added to `State`. The `realize_gate` definition and other mandatory function definitions in Amaro omit the fixed parameters from the left-hand side of the assignment for simplicity, and the parameters appear as unbound variables in the expression on the right-hand side. In this case, `realize_gate` calls the `GateRealization` constructor on each edge in the QPU `Arch` between the mapping locations of the two arguments of the CX gate in a functional programming style, using the higher-order function `map`.

TransitionInfo Block. The `TransitionInfo` block defines the transition relation between qubit maps of adjacent states. The first line defines the `Transition` datatype. In this case, the transitions are described by a pair of QPU locations to which a SWAP gate is applied, so once again the datatype has one field which is a pair of elements of type `Loc`.

```
Transition{edge : (Loc, Loc)}
```

We constrain the pairs to be adjacent in the connectivity graph of the target QPU with the definition of another mandatory function `get_transitions`. The function `get_transitions` has two implicit parameters, a QPU `Arch` and a state `State`, and returns a list with elements of type `Transition`. In this case, `get_transitions` constructs a `Transition` for each edge in the QPU `Arch`.

```
// get_transitions : (Arch, State) → List[Transition]
get_transitions = map(|x| → Transition{edge = x}, Arch.edges())
```

Then, the next line defines a function `apply` which describes the action of a transition on a qubit map, which is to swap the circuit qubits mapped to those locations. The Amaro standard library function `value_swap` simplifies this definition.

```
// apply : (Transition, QubitMap) → QubitMap
apply = value_swap(QubitMap, Trans.edge.(0), Trans.edge.(1))
```

Finally, the cost definition says each non-trivial transition (i.e. inserted SWAP gate) has a cost of 1.

```
// cost : Transition → Real
cost = if Trans == IdTrans then 0.0 else 1.0
```

```

RouteInfo:
  GateRealization{edge : (Loc,Loc)}
  routed_gates = [CX]
  realize_gate = map(|x| → GateRealization{edge = x},
    Arch.edges_between(State.map[Gate.qubits[0]], State.map[Gate.qubits[1]]))

TransitionInfo:
  Transition{edge : (Loc,Loc)}
  get_transitions = map(|x| → Transition{edge = x}, Arch.edges())
  apply = value_swap(QubitMap, Trans.edge.(0), Trans.edge.(1))
  cost = if Trans == IdTrans
    then 0.0
    else 1.0

```

Fig. 5. The Amaro definition of the NISQMR problem

This simple example demonstrates the general structure of an Amaro program: defining data structures and functions which form an operational description of the constraints of a QMR problem. Amaro offers a few other features that were not needed here but do appear in other QMR problem definitions. These include another optional block for defining the expected labels on the QPU graph edges and vertices (e.g., to say which locations are reserved for storing magic states in SCMR) and built-in utilities for common graph algorithms like path-finding and Steiner tree construction.

2.4 Generating a QMR Compiler from an Amaro Program

Given an Amaro program P , we can generate a QMR compiler $Comp_P$ for the QMR problem it specifies (recall Fig. 1). Specifically, given a circuit and a QPU graph, $Comp_P$ should construct a minimal cost sequence of device states that solves the QMR problem. We generate the compiler by *instantiating* the generic Amaro QMR solver for the given Amaro program P . The algorithm underlying the Amaro solver iteratively constructs a sequence of device states satisfying the definitions in P —like the ones in Fig. 3 and Fig. 4—that execute all gates in a given circuit on the given QPU.

QMR problems are generally NP-hard, and it has been shown that optimal approaches (e.g., using SAT solvers) do not scale to large circuits and devices. Amaro is an approximate algorithm: In each iteration, Amaro constructs a *maximal state*, one in which no more gates can be executed in parallel. The states depicted in Fig. 3 and Fig. 4 are all examples of maximal states.

We identify a key property of QPUs, which we call *monotonicity*, that allows us to construct maximal states without requiring search (all practical QPUs are monotonic). We also identify the *non-interference* property—executing one gate does not block another—which allows us to construct *maximum* states without requiring search. By statically analyzing Amaro programs, we can identify non-interference and disable the search process, thus speeding up the solving. For example, NISQMR is non-interfering while SCMR is not, because the paths of one gate can block another gate’s paths.

As we demonstrate later, (1) we used Amaro to generate a diverse range of QMR compilers for noisy and fault-tolerant QPUs; (2) the generated compilers are competitive with specialized QMR solvers in terms of solution quality and runtime.

3 The Abstract QMR Problem

In this section, we formally define the abstract QMR problem.

3.1 Circuits and QPUs

We begin by defining the two inputs to the QMR problem: circuits and QPUs. We view a circuit as a sequence of applications of quantum gates. Throughout, we fix a universe of gates *Gates* that can appear in a circuit and a universe of qubits *Qubits* that they can be applied to.

Definition 3.1 (Instruction). An instruction $g(\bar{q})$ is a quantum gate $g \in \text{Gates}$ acting on a list of qubits $\bar{q}$ drawn from *Qubits*. We denote the set of all instructions as *Instrs*.

Definition 3.2 (Quantum circuit). A circuit is an indexed sequence $g_1(\bar{q}_1), \dots, g_k(\bar{q}_k)$ of instructions. (We will often drop the $\bar{q}_i$ and refer to the instruction simply as g_i .)

Gates in a circuit which act on the same qubit(s) must be executed in order. This naturally gives us a partial order on circuit instructions and a notion of a topological layering of a circuit.

Definition 3.3 (Gate dependency). Instruction $g_j(\bar{q}_j)$ directly depends on instruction $g_i(\bar{q}_i)$ if $i < j$ and some qubit appears in both—that is, $\bar{q}_i \cap \bar{q}_j \neq \emptyset$. *Dependency* is the transitive closure of direct dependency and is denoted $g_i < g_j$.

Definition 3.4 (Circuit layer). A layer is a set of circuit instructions with no dependencies between them. The *front layer* is the set of instructions that do not depend on any instruction in the circuit.

Example 3.5. Consider the circuit in Fig. 3b. The direct dependencies in this circuit—also the only dependencies—are $g_0 < g_2$, $g_0 < g_3$, $g_1 < g_2$, and $g_1 < g_3$. This circuit has the front layer $\{g_0, g_1\}$.

Next is the representation of a QPU. Our abstraction for a QPU is a set of locations *Locs* (typically denoting device qubits) and device instructions *DInstrs*.

Example 3.6. For the NISQMR problem, we represent a QPU with a connectivity graph. The set *Locs* is the graph vertices, which represent physical qubits. The device instructions *DInstrs* are the edges of the graph. For the SCMR problem, we represent a QPU with a grid graph. The device instructions *DInstrs* are (certain) paths in the graph.

3.2 Device State Machines

To model a time step of execution on a given device, we define the notion of a QPU *state*. In essence, a QPU state is the implementation of a set of gates which can be executed in parallel while satisfying the constraints of the device. We have seen examples of QPU states in Sec. 2.2. As seen in the tabular representations in Figs. 3c, 4c and 4d, the state of a QPU consists of two parts. One piece of a QPU state is a qubit *map*, which assigns circuit qubits to locations. The other is an assignment of instructions in the circuit to device instructions—a set of gate *routes*. Note that these are both *partial* functions. The qubit map is only defined on qubits that appear in the circuit, and the gate route is only defined on a set of circuit instructions that can be executed in parallel.

Definition 3.7 (Device state). The state of a QPU is a pair of injective partial functions:

- $\text{map} : \text{Qubits} \rightarrow \text{Locs}$
- $\text{routes} : \text{Instrs} \rightarrow \text{DInstrs}$

Definition 3.8 (Device state machine). We call a tuple $(\text{Real}, \rightarrow_c)$ a QPU state machine, where

- *Real* is a predicate on device states which determines whether a state is a physically realizable implementation of the circuit instructions in its gate routing.
- $\rightarrow_c$ is a transition relation between states $s \rightarrow_c s'$ where c denotes the cost of the transition.

Example 3.9. For the NISQMR problem, a state is realizable if every two-qubit gate acts on adjacent qubits. Let $s = (\text{map}, \text{routes})$ be a state of a NISQMR QPU. The predicate *Real*(s) holds if and only if every circuit instruction $\text{CX } q_i q_j$ in the domain of routes is assigned to the edge $(\text{map}(q_i), \text{map}(q_j))$.

For SCMR, a state is realizable if gate routing paths are (vertex-)disjoint. Let $s = (\text{map}, \text{routes})$ be a state of an SCMR QPU. The predicate $\text{Real}(s)$ holds iff the following three conditions are met

- every CX instruction $\text{CX } q_i \ q_j$ in the domain of routes is assigned to a path from a vertical neighbor of $\text{map}(q_i)$ to a horizontal neighbor of $\text{map}(q_j)$.
- every T instruction $T \ q_i$ in the domain of routes is assigned to a path from a vertical neighbor of $\text{map}(q_i)$ to a horizontal neighbor of a magic state qubit.
- no vertex appears in two distinct paths

Example 3.10. For NISQMR, the transitions are SWAP operations between adjacent physical qubits on the QPU, along with an identity transition which leaves the qubit map unchanged. The transition relation $\rightarrow_c$ relates any pair of states with qubit maps that differ by swapping the locations of a pair of qubits along an edge of the device. The cost of each nontrivial transition is 1.

For SCMR, the qubit map is fixed throughout, so the only transition is the trivial identity transition. We assign a constant transition cost of 1.

3.3 Mapping and Routing Solutions

The goal of QMR is to find a valid sequence of states that routes all the gates and minimizes cost.

Definition 3.11 (Mapping & Routing Solution). Given a circuit C and QPU state machine $(\text{Real}, \rightarrow_c)$, a *mapping and routing solution* is a sequence of states

$$s_1 \rightarrow_{c_1} \cdots \rightarrow_{c_{k-1}} s_k$$

such that

- $\text{Real}(s_i)$ holds for all $i \in [1, k]$;
- each instruction g in C appears in exactly one state, which we denote $\text{state}(g)$; and
- if g and g' are instructions s.t. $g < g'$, then $\text{state}(g)$ appears before $\text{state}(g')$ in the solution.

The cost of a mapping and routing solution is simply the sum of the transition costs, $\sum_{i=1}^{k-1} c_i$. The goal is to find a mapping and routing solution of minimum cost.

Example 3.12. It is straightforward to verify the solution shown in Fig. 3c is a mapping and routing solution for the input shown in Fig. 3 when Real and $\rightarrow_c$ are chosen for the NISQMR problem as described in Theorem 3.9 and Theorem 3.10. The cost of this solution is 1.

4 A Language for Specifying QMR Problems

In this section, we describe the Amaro language for specifying QMR problems.

4.1 Amaro Language Design

The goal of Amaro is to define a QMR problem by specifying a family of similar QPUs with related constraints on their states and transitions. For example, the SCMR problem is defined by grid graphs of error-corrected qubits (refer to Fig. 4a), while the NISQMR problem is defined by noisy qubits connected by edges (refer to Fig. 3a). Notice that we represent a target QPU with a graph in both cases. Generally, graphs are the main abstraction of a quantum computer used in formulating QMR problems. Consequently, graphs are a core primitive in Amaro. A program in Amaro defines a QMR problem by describing how to interpret a graph as a QPU state machine, filling in the state predicate Real and transition relation $\rightarrow_c$.

Solution Generation. Ultimately, we want to efficiently *generate* solutions for a given circuit and QPU, not just verify that a solution is valid. With this aim in mind, Amaro is designed such that Real and $\rightarrow_c$ are defined “constructively,” with functions like `realize_gate` and `get_transitions` that

$g \in \text{Gates}$ (gates)	$ABlk ::= \varepsilon$
$x \in \mathcal{V}$ (variables)	$ \text{ArchInfo: Arch}\{\bar{x}:\bar{\tau}\}; \text{get_locations} = e$
$\tau ::= \text{Loc} \mid \text{Int} \mid \text{Float} \mid \text{Bool} \mid$ $ \text{List}[\tau] \mid \tau \times \tau \mid \tau \rightarrow \tau \mid S$ $ \text{ArchT} \mid \text{InstrT} \mid \text{Qubit} \mid \text{StateT}$	$SBlk ::= \varepsilon \mid \text{StateInfo: cost} = e$
$P ::= RBlk \mid TBlk \mid ABlk \mid SBlk$	$F ::= \text{edges} \mid \text{all_paths} \mid \text{steiner_trees} \mid$ $ \text{push} \mid \text{map} \mid \text{fold} \mid \text{concat} \mid \dots$
$RBlk ::= \text{RouteInfo:}$ $\quad \text{GateRealization}\{\bar{x}:\bar{\tau}\};$ $\quad \text{routed_gates} = \bar{g};$ $\quad \text{realize_gate} = e;$	$v ::= r \in \mathbb{R} \mid n \in \mathbb{N} \mid \text{str} \mid \text{loc}(v) \mid (v, v)$ $ x \rightarrow e \mid S\{\bar{x} = \bar{v}\} \mid [\bar{v}]$
$TBlk ::= \text{TransitionInfo:}$ $\quad \text{Transition}\{\bar{x}:\bar{\tau}\};$ $\quad \text{get_transitions} = e;$ $\quad \text{apply} = e;$ $\quad \text{cost} = e$	$e ::= x \mid \text{Arch} \mid \text{Instr} \mid \text{State} \mid \text{Trans}$ $ \text{QubitMap} \mid \text{State.map} \mid \text{State.route}$ $ \text{Instr.gate_type} \mid \text{Instr.qubits}$ $ \text{loc}(e) \mid e \otimes e \mid e.x \mid [\bar{e}] \mid e[e] \mid e \, e$ $ (e, e) \mid \text{proj}_i e \mid \text{if } e \text{ then } e \text{ else } e \mid F(\bar{e})$ $ \text{GateRealization}\{\bar{x} = \bar{e}\}$ $ \text{IdTrans} \mid \text{Transition}\{\bar{x} = \bar{e}\}$

Fig. 6. The grammar of Amaro programs

output lists of valid options, as opposed to declaratively in terms of constraints. This design decision enables a natural strategy to find a solution for a given circuit and QPU.

With the information in an Amaro program, we can associate a circuit and graph with a set of valid mapping and routing solutions and determine the cost of each solution. In other words, when we write a program P in Amaro, its semantics is treated as a set of tuples where

$$(C, A, Sol) \in \llbracket P \rrbracket$$

means that circuit C has a mapping and routing solution Sol (Theorem 3.11) for the QPU represented by graph A .

4.2 The Amaro Grammar

The syntax of Amaro is presented in Fig. 6. Here, P is an Amaro program, and the non-terminals with “-Blk” names are the definition blocks which comprise an Amaro program. There are four of these blocks: the mandatory `RouteInfo` and `TransitionInfo` blocks and the optional `ArchInfo` and `StateInfo` blocks. The `ArchInfo` block defines any labels on the QPU graph, such as the locations of magic state qubits in SCMR. The `StateInfo` block defines a cost function on states, for convenience when assigning a cost to a step is a more natural formulation than a transition cost.¹

Within each block are definition lines, which have a keyword on the left-hand side and one of three options on the right. First, there is the `routed_gates` line, for which the right-hand side is a list of gate identifiers (e.g. “cx” or “T”) drawn from *Gates*. Second, there are data definition lines, where the right-hand side is a list of field names (drawn from the set of identifiers $\mathcal{V}$) and their types, which define the datatype associated with the block. Finally, there are function definition lines, where the right-hand side is an expression from a standard functional expression language defining a function on fixed implicit arguments.

4.3 Amaro Semantics

The semantics of Amaro programs are presented in Fig. 7. Together, an Amaro program P and a labeled graph A define a QPU state machine $(Real, \rightarrow_c)$. We use the judgement $P, A \vdash \psi$ to mean

¹Any QMR problem with costs on states can be expressed with transition costs alone, so this block is purely for ease of use.

$$\begin{array}{c}
\text{[FULLPROG]} \frac{\begin{array}{c} \text{Sol} = s_1 \rightarrow_{c_1} \dots \rightarrow_{c_{k-1}} s_k \quad P, A \vdash \forall i \in [1, k]. \text{Real}(s_i) \\ P, A \vdash \forall i \in [1, k-1]. (s_i \rightarrow_{c_i} s_{i+1}) \quad \text{DR}(C, \text{Sol}) \end{array}}{(C, A, \text{Sol}) \in \llbracket P \rrbracket} \\
\\
\begin{array}{cc}
\text{[REALEMPTY]} \frac{\text{range}(\text{map}) \subseteq \text{Locs}}{P, A \vdash \text{Real}((\text{map}, \emptyset))} & \text{[REALINS]} \frac{\begin{array}{c} P, A \vdash \text{Real}((\text{map}, \text{routes})) \\ r \in \llbracket P.\text{realize_gate} \rrbracket (A, (\text{map}, \text{routes}), \text{ins}) \end{array}}{P, A \vdash \text{Real}((\text{map}, \text{routes}[\text{ins} \mapsto r]))} \\
\\
\text{[TRANSITION]} \frac{\begin{array}{c} s = (\text{map}, \text{routes}) \quad s' = (\text{map}', \text{routes}') \\ t \in \llbracket P.\text{get_transitions} \rrbracket (s, A) \quad \text{map}' = \llbracket P.\text{apply} \rrbracket (t, \text{map}) \quad c = \llbracket P.\text{cost} \rrbracket (t) \end{array}}{P, A \vdash s \rightarrow_c s'}
\end{array}$$

Fig. 7. Amaro semantics. The premise $\text{DR}(C, \text{Sol})$ is a predicate that checks that every instruction of the circuit is in the step sequence and the dependencies are respected (latter two conditions of Theorem 3.11).

the formula ψ holds for the definitions of Real and $\rightarrow_c$ given by P and A . The **FULLPROG** rule defining the semantics of a full program just rephrases the definition of the abstract QMR problem (Theorem 3.11) for the QPU state machine defined by the program and graph.

Definition 4.1 (Amaro semantics). The semantics of an Amaro program P , written $\llbracket P \rrbracket$, is the set of all tuples satisfying the inference rules in Fig. 7.

The other rules use the semantics of expressions $\llbracket e \rrbracket$ as functions. Formally, we equip the Amaro expression language with a standard deterministic small-step operational semantics. For an expression e with free variables $\bar{x} = \text{fv}(e)$, we define $\llbracket e \rrbracket$ to be the partial function that, on inputs $\bar{v}$, evaluates $e[\bar{x} \mapsto \bar{v}] \rightarrow^* w$ and returns w . If $e[\bar{x} \mapsto \bar{v}]$ gets stuck, then $\llbracket e \rrbracket(\bar{v})$ is undefined.

We also equip Amaro with a simple type system to ensure that the supplied code expects the correct arguments and computes the correct types. Specifically, the expressions in the mandatory function definitions have the following signatures:

```

P.realize_gate  : ArchT, StateT, StateT → List[GateRealization]
P.get_transitions : StateT, ArchT → List[Transition]
P.apply        : Transition, QubitMap → QubitMap (where QubitMap = Qubit → Loc)
P.cost         : Transition → Float

```

See Appendix C in the extended version of this paper [38] for more details on the Amaro type system and semantics.

The **REALEMPTY** and **REALINS** rules formalize the notion that a realizable state is the result of inductive application of the `realize_gate` function. The set *Locs* is the vertices of A if not overridden by a `get_locations` definition.

The **TRANSITION** rule relates the transition relation $\rightarrow_c$ to the `TransitionInfo` block of the program. A pair of states (s, s') satisfies the transition relation if the qubit map of s' results from applying one of the transitions returned by `get_transitions`. The cost of the transition is given by evaluating the cost function from P on the transition.

5 Solving QMR Problems via Maximal-State Construction

In this section, we describe the Amaro solver for the QMR problems defined in the Amaro language. For a given Amaro program P , we instantiate the solver as Amaro_P . Amaro_P takes a circuit and

graph, and returns a valid mapping and routing solution of the form

$$s_1 \rightarrow_{c_1} \cdots \rightarrow_{c_{k-1}} s_k$$

Amaro iteratively constructs a sequence of states while ensuring that each state is *maximal*, meaning that it cannot be extended to a realizable state that routes any additional instructions. Since different choices of initial map can yield solutions of differing costs, we repeat this iterative construction with several different initial maps and return the best solution, as described in Sec. 5.3.

Given the general hardness of solving QMR problems, Amaro is not guaranteed to find an optimal solution. Indeed, as we shall describe, we employ *simulated annealing* [31]—a classic randomized search algorithm—in different parts of the algorithm to steer the search towards better solutions, inspired by prior work in QMR and other quantum compilation problems [40, 46]. See Appendix B in the extended version of this paper [38] for details on the parameters we choose to instantiate our simulated annealing search.

5.1 The Amaro Solver: A High-Level View

The high-level steps of the Amaro solver are shown in Algorithm 1. The algorithm starts with an empty solution, *Sol*. In each iteration, the algorithm constructs a state to add to the solution, *Sol*. The key invariant Amaro maintains is that each state s_i that is added to *Sol* is realizable and that every consecutive pair of states s_i, s_{i+1} in *Sol* is such that $s_i \rightarrow_{c_i} s_{i+1}$.

Let us walk through the solver step by step:

Line 3: Amaro begins by constructing an initial qubit map *map*; in principle, this can be a random map. In Sec. 5.3, we discuss a more sophisticated strategy that converges on a suitable initial map through repeated applications of Algorithm 1.

Line 5: In each iteration, we get the *front layer* of instructions from the circuit (see Theorem 3.4). Recall that this is the set of all independent instructions in the prefix of the circuit; being independent, these instructions can soundly be routed in parallel if the device permits.

Line 6: Next, the algorithm tries to route as many of the instructions in the front layer as possible. This is done by a process we call *maximal-state construction*, which we describe in detail in Sec. 5.2. The constructed maximal state is added to *Sol* and the routed instructions are removed from the circuit.

Line 8: The next step is to update the map for the next state. This is done by calling the `get_transitions` function, which returns a set of valid transitions from the current state. The algorithm then selects one of the transitions and updates the map accordingly. We describe a strategy for selecting the next transition in Sec. 5.3.

The process repeats until all instructions in the circuit have been routed.

Example 5.1. As an example, we walk through how this algorithm could be used to construct the NISQMR solution from Sec. 2.2 (Fig. 3c). We begin by choosing the initial qubit map depicted, which maps q_i to p_i . Then, we find the front layer of this circuit, which includes two instructions: $\text{cx } q_0 q_1$ and $\text{cx } q_2 q_3$. Both of these instructions can be routed under our qubit map, and so they are included in the maximal state. Routing the entire front layer is the best-case maximal state.

Next, we must choose from the set of four transitions—a SWAP along one of the three edges or the identity. Not all choices are equal. If we choose the identity, we fail to make progress in executing the circuit. At the next iteration of the loop, the front layer consists of the instructions $\text{cx } q_1 q_3$ and $\text{cx } q_0 q_2$. Neither of these are executable, so the maximal state has an empty gate route. On the other hand, if we choose to SWAP along the edge (p_1, p_2) as depicted in Fig. 3c, we obtain another

Algorithm 1 Amaro_P algorithm for finding mapping and routing solutions

```

1: procedure AmaroP(graph  $A$ , circuit  $C$ )
2:   Initialize an empty state sequence  $Sol$ 
3:   Create initial map, map ▷ Sec. 5.3
4:   while  $C \neq \emptyset$  do ▷ note we remove routed instructions from  $C$ 
5:     Compute  $E$ , the front layer of instructions in  $C$ 
6:     Construct a maximal state  $s$  for map and  $E$  ▷ Sec. 5.2
7:     Append  $s$  to  $Sol$  and remove instructions routed in  $s$  from  $C$ 
8:     Get the next map  $map'$  from the valid transitions given by  $s$ ,  $A$  and  $P$  ▷ Sec. 5.3
9:     Set  $map \leftarrow map'$ 
10:  return  $Sol$ 

```

maximum set containing both of the candidate instructions. At this point, the algorithm terminates as the full circuit has been routed.

Correctness and Termination. Theorem 5.2 says that any solution returned by Amaro is a valid mapping and routing solution. Moreover, Amaro terminates with a solution as long as the input program and graph do not define sets of states closed under the transition relation that cannot make progress in execution, as defined precisely in Theorem 5.3. This condition rules out pathological examples like a `realize_gate` function that always returns an empty list. All of our case study problem instances satisfy the termination condition. Nevertheless, despite the guarantee of eventual termination, Amaro sometimes fails to find a solution within fixed resource bounds, as shown in the plots in Sec. 7.

THEOREM 5.2 (Amaro_P SOUNDNESS). *Let Sol be the solution returned by Amaro_P on input (A, C) . Then, $(C, A, Sol) \in \llbracket P \rrbracket$.*

THEOREM 5.3 (Amaro_P TERMINATION). *Let $(Real, \rightarrow_c)$ be the QPU state machine defined by an Amaro program P and graph A . Suppose that for any Real state s and instruction g , there exists a Real state s' reachable from s under $\rightarrow_c$ such that g is in the gate route of s' . Then, Amaro_P terminates on input (A, C) for any circuit C .*

5.2 Maximal-State Construction

We now describe the Amaro strategy for constructing maximal states. In the process, we define two conditions on QPU state machines: *monotonicity* and *non-interference*. In Theorem 5.7, we use these definitions to classify the QPU state machines for which we can efficiently find a maximal state and those for which the maximal state is unique.

First, we precisely define a maximal state.

Definition 5.4 (Maximal State). Consider the states routing instructions from a circuit layer E . A realizable state $s = (\text{map}, \text{routes})$ is *maximal* if there is no realizable super-state $s' = (\text{map}, \text{routes}')$ such that $\text{routes} \subset \text{routes}'$, where $\subset$ denotes strict inclusion.

We observe that, for the QMR problems we consider, we can find a maximal state for a set of parallel instructions with one pass through the instructions, as shown in Algorithm 2: the algorithm simply iterates through the instructions, calls `realize_gate` for each, and if the result is a non-empty set of gate realizations, chooses one to add to the gate route. Algorithm 2 is an efficient procedure that always produces maximal states for “reasonable” settings like our case study problems, but does not find a maximal state for any arbitrary QPU state machine.²

²In fact, we cannot expect a tractable algorithm for finding maximal states without introducing restrictions on the *Real* predicate. For a (contrived) example, we could define a family of QPUs for the boolean satisfiability problem. In this family,

Algorithm 2 Constructing a maximal state

```

procedure ROUTE-ONE-PASS(graph  $A$ , program  $P$ , layer  $E$ , qubit map  $\text{map}$ )
  Initialize the state  $s$  with qubit map  $\text{map}$  and empty gate route set  $\text{routes}$ .
  for each instruction  $g$  in  $E$  do
    Let  $\text{route-candidates} = \llbracket P.\text{realize\_gate} \rrbracket(A, s, g)$ 
    if  $\text{route-candidates}$  is non-empty then
      Set  $\text{routes}(\text{ins}) = r$  for some  $r \in \text{route-candidates}$ 
  return  $s$ 

```

Algorithm 3 Search for the best maximal state

```

procedure ROUTE(graph  $A$ , program  $P$ , layer  $E$ , qubit map  $\text{map}$ )
  Initialize the state  $s$  with qubit map  $\text{map}$  and empty gate route set  $\text{routes}$ .
  Let  $\Sigma$  be the set of all permutations of  $E$ 
   $\sigma^* \leftarrow \text{argmax}_{\sigma \in \Sigma} |\text{ROUTE-ONE-PASS}(A, P, \sigma, \text{map})|$ 
  return  $\text{ROUTE-ONE-PASS}(A, P, \sigma^*, \text{map})$ 

```

5.2.1 Monotonicity and Maximality. We define the notion of *monotonic Real* predicates as a condition that is strict enough to ensure Algorithm 2 finds a maximal state, but permissive enough to include any realistic QPU state machine. Indeed, all of our case study problems are monotonic.

Definition 5.5. (Monotonicity) A predicate *Real* is *monotonic* if whenever a state $s = (\text{map}, \text{routes})$ is realizable, so is any sub-state $s' = (\text{map}, \text{routes}')$ with $\text{routes}' \subseteq \text{routes}$.

For some QMR problems, we reach the *unique* maximal state with Algorithm 2 regardless of which order we iterate over the layer or which gate realization we select. We categorize these cases as *non-interfering*. The intuition behind the definition of non-interference is that iteration order is irrelevant when routing one instruction in a state does not prevent the routing of another. The NISQMR problem is non-interfering, but SCMR is not because the path of one instruction can occupy vertices and prevent the routing of another.

Definition 5.6. (Non-interference) A predicate *Real* is *non-interfering* if whenever $s = (\text{map}, \text{routes})$ and $s' = (\text{map}, \text{routes}')$ are *Real* states with the same qubit map which route disjoint sets of instructions, the combined state $(\text{map}, \text{routes} \cup \text{routes}')$ is also *Real*.

THEOREM 5.7. *The procedure ROUTE-ONE-PASS produces a realizable state for any QPU state machine. If the predicate Real is monotonic, the resulting state is maximal. If Real is also non-interfering, then the maximal state is unique in terms of routed instructions.*

If the predicate *Real* is *not* non-interfering, then maximal states can have different sizes. We search for the iteration order which maximizes the number of routed instructions. The full maximal step procedure with this case is shown in Algorithm 3. Notice how the algorithm is identical to Algorithm 2 except for the addition of the search over permutations of the instructions. We implement this search with simulated annealing. Each search step of simulated annealing randomly swaps the position of two instructions in the order. The cost of a candidate permutation is the number of routed instructions in the constructed maximal state.

each QPU state machine corresponds to a boolean formula, and the *Real* predicate interprets a subset of the layer as a variable assignment, returning true on a state if the routed instructions constitute a satisfying assignment to the formula.

5.3 Selecting an Initial Map and Transitions

Ultimately, the goal of Amaro is to find a low-cost solution. We now describe how we select the initial qubit map and transitions with the goal of minimizing solution cost.

Initial Map. Our algorithm searches for the initial qubit map that yields the best solution. Instantiating map with any qubit map will yield some valid mapping and routing solution. However, different choice of initial qubit maps often lead to solutions of different cost, as we saw with the two examples in Figs. 4c and 4d. We explore the space of possible initial qubit maps to find the one which results in the lowest cost mapping and routing solution. The search for a minimum cost solution over initial qubit maps is implemented via simulated annealing. Each search step of simulated annealing modifies the initial qubit map by exchanging the mapping locations of two qubits or moving a qubit into an unused location, then applies Algorithm 1 with this choice of map. The cost of an initial qubit map in this simulated annealing search is the cost of the final mapping and routing solution.

Transitions. Recall that the Amaro algorithm maintains a sequence of states Sol which is a valid solution to the QMR problem. In each iteration, the algorithm constructs a state to add to the solution, Sol , along with a transition to the next state. There are many ways to choose the next transition; we simply choose a transition t that maximizes the next state's size in terms of number of routed instructions:

$$\max_t |\text{ROUTE}(A, P, E', \llbracket P.\text{apply} \rrbracket(t, m))| - \llbracket P.\text{cost} \rrbracket(t) \quad (1)$$

Here, E' is the front layer of instructions in the circuit after removing instructions routed in the current, as well as previous, states.

5.4 Key Optimizations

We now describe some of the key optimizations that improve the performance of Amaro.

Static Analysis for Non-Interference. By Theorem 5.7, if $Real$ is non-interfering, the search over the space of maximal states in Algorithm 3 is unnecessary because there is a unique maximal state. We identify non-inference in practice through a simple static analysis of the Amaro program. We claim that if the `realize_gate` definition does not contain the subexpression `State.route`, then the resulting $Real$ predicate must be non-interfering. Notice that a counterexample to non-interference implies the existence of an instruction g and two states $s = (\text{map}, \text{routes})$ and $s' = (\text{map}, \text{routes}')$ such that $\llbracket P.\text{realize_gate} \rrbracket$ is nonempty for s and empty for s' . However, if `realize_gate` definition does not contain the subexpression `State.route`, then it must evaluate to the same result regardless of the input gate route. Therefore, we can soundly over-approximate the property of non-interference by traversing the AST of the `realize_gate` definition in search of the subexpression `State.route`. If there is no match, we safely assume non-interference.

Transition Selection Optimizations. We close with some additional practical considerations in transition selection. For one, there are some scenarios where no transition enables execution of any instructions. For example, consider a NISQMR problem where the front layer `CX` instructions act on qubits that are more than one `SWAP` away from adjacent. In order to continue to make progress, we choose the transition that minimizes the distance between qubits acted upon by instructions in the front layer. Second, we weigh routed instructions by *criticality*, following a strategy introduced in prior QMR work [24, 40]. The criticality of an instruction is the length of the longest sequence of dependent instructions which begins with g . We prioritize critical instructions to avoid a long “tail” of sparse states as the instructions on the critical path are executed in sequence.

6 Qubit Mapping & Routing Case Studies

To demonstrate the practical utility and flexibility of our approach, we present seven case studies that reflect real-world qubit mapping and routing challenges drawn from the literature. These examples span different types of quantum hardware and programming models, each with unique constraints. We select problems (described below) which demonstrate that our design supports:

- (1) **Diverse hardware:** There are multiple competing implementations of the physical qubit including superconducting circuits, Rydberg neutral atoms, and trapped ions. Physical characteristics of underlying hardware impose constraints on mapping and routing.
- (2) **Devices with & without error correction:** We are currently at an inflection point for quantum error correction. Devices that are currently accessible to the public do not implement error correction. However, in recent years we have seen early experimental demonstrations of error correction. Emerging error-correcting codes bring new constraints that affect how qubits can be moved, measured, and interacted with. Our framework is designed to handle both modes—pre- and post-error correction—enabling developers to experiment with evolving designs and co-optimize across hardware, QEC strategies, and applications.
- (3) **Discrete & continuous cost functions:** In some cases, the appropriate metric of solution quality is discrete, like the number of added SWAP gates or the total number of time steps; in others, the cost function is continuous, like the probability of successful computation.

6.1 Architectures without Error Correction

NISQMR. The first case study is the NISQMR problem described in Sec. 2.2. Two-qubit gates are only allowed between physical qubits which are adjacent in the connectivity graph. SWAP gates are inserted to transform the mapping. NISQMR is the relevant QMR problem when targeting hardware with fixed position qubits (like superconducting circuits) and no error correction. This case models today’s widely accessible superconducting hardware and reflects the default compilation mode in most current quantum toolchains. The Amaro description of NISQMR is the example shown in Fig. 5.

Variable-Error NISQ (NISQ-VE). On real quantum hardware, not every link is equally reliable. Error rates between different two-qubit gates can differ by an order of magnitude [59]. When performing mapping and routing, we should prefer to make use of the two-qubit gates with lower error rates. When we take variation into account, the optimization objective changes to direct maximization of the probability of successful computation, rather than minimization of the number of SWAP gates

We can easily capture this version of the problem, which we call NISQ-VE (for variable error) in the abstract QMR framework. The changes to the Amaro definition are shown in Fig. 8. Lines identical to the original NISQ are elided. Note the addition of the nontrivial ArchInfo block, which carries the data of the reliability of each edge. In order to use it as an additive cost, this reliability is represented as $-\log(p_{succ})$ where p_{succ} is the probability of error-free gate execution.

We use this data to redefine the cost of a SWAP by looking up the success rate of the corresponding edge. Likewise, we add a StateInfo block to define the cost of a step as a sum over the success rates of each of the implemented gates.

Trapped-Ions (TIQMR). Trapped-ions are an alternative candidate hardware platform. Each qubit on a trapped-ion QPU is implemented as an atomic ion which is trapped with an electromagnetic field [48]. Two-qubit gates can be performed between any pair of qubits in the same trap. However, each trap is limited to 10s of qubits. Therefore, proposed trapped-ion architectures consist of several interconnected traps. To perform a two-qubit gate between qubits in different traps, we need to use

```

RouteInfo:
  ...
TransitionInfo:
  ...
  cost = if Trans = IdTrans
    then 0.0
    else Arch.edge_cost[Trans.edge.(0)][Trans.edge.(1)]
ArchInfo:
  Arch{edge_cost : List[List[Float]]}
StateInfo:
  cost=fold(0,
    |acc,x| → acc+x,
    map(|x| → Arch.edge_cost[State.map[x.qubits[0]]][State.map[x.qubits[1]]],
      State.route))

```

Fig. 8. The Amaro definition of the NISQ-VE QMR problem

shuttling operations to physically move qubits from one trap to another. The typical cost function for trapped-ion QMR (TIQMR) is the total added time for shuttling operations [3, 42].

Reconfigurable Atom Arrays (RAA). Another competing hardware platform is the neutral atom quantum computer. Each qubit is a Rydberg neutral atom which is trapped in a two-dimensional array of optical tweezers [5], enabling programmable qubit layouts. Two-qubit gates are executable between qubits within a target radius of one another. For long-range interactions, atomic qubits can be repositioned over the course of computation by modulating the tweezers. However, the movements are slow and constrained in direction [56, 57]. An entire row or column of the qubit array must be shifted in parallel, and the relative positions of rows and columns is fixed. In reconfigurable atom array QMR, each movement operation is associated with an error rate, and the cost function is the probability of successful execution. As in NISQ-VE, we convert this to an additive cost by assigning each operation a cost of $-\log(p_{succ})$ where p_{succ} is the probability of error-free execution.

6.2 Architectures With Error Correction

SCMR. We begin our study of QMR in the presence of error correction with the SCMR problem from Sec. 2.2. In this setting, two-qubit gates and T gates are implemented by routing paths between locations on the architecture. Simultaneous paths cannot cross (must be vertex-disjoint). The cost function for this problem is the total number of states.

Interleaved Logical Qubits (ILQ). The flexibility of configuration in space for Rydberg atoms can be used to implement an *interleaved* architecture where logical qubits are “stacked” on top of one another [54]. In routing two-qubit gates, there is a distinction between two-qubit gates applied to qubits in the same stack (intra-stack gates) and two-qubit gates applied to qubits in different stacks (inter-stack gates). Inter-stack two qubit gates are implemented via lattice surgery and take time that scales with the number of physical qubits used per logical qubit (i.e. the code distance), whereas the intra-stack gate takes constant time, independent of the size of logical qubit. Therefore the cost function assigns a cost of 1 to a state with only intra-stack gates and a cost equal to the code distance to other states.

Multi-Qubit Lattice Surgery (MQLSS). Some quantum computing models, such as Pauli-based computation, require multi-qubit measurements involving arbitrary sets of qubits. The lattice surgery procedure used to implement two-qubit gates on surface code logical qubits can be extended to support multi-qubit measurements [37, 54]. Multi-qubit measurements are implemented using

branching lattice surgery paths, forming tree structures rather than linear routes. This case study generalizes the routing model, helping developers prototype and evaluate alternative computational models for fault-tolerant execution. The cost of a solution is the total number of states.

7 Implementation and Empirical Evaluation

We implemented Amaro as a Rust library (~6500 LoC). A QMR problem P defined in Amaro is translated to Rust and compiled with the library, generating a binary `AmaroP`.³ This binary takes a circuit and a QPU graph as input and produces a mapping and routing solution. The generated compiler leverages parallelism by default, instantiating one run of Amaro search per allotted CPU core and returning the best result.

We aim to answer the following empirical research question for each of our case studies:

Q1 How do our generated compilers compare to problem-specific state-of-the-art approaches in terms of solution quality?

Q2 How long do our generated compilers take to converge to a solution?

We also evaluate the impact of individual algorithm choices in the Amaro solver.

Q3 How do initial qubit map search and maximal state search contribute to solution quality?

Benchmark circuits. For each case study, we benchmark compilers with the suite of 243 application circuits collected by Molavi et al. [40], which subsumes that of Zulehner et al. [68]. This suite contains arithmetic circuits derived from the RevLib suite [63], programs written in the Quipper [17] and ScaffoldCC [25] quantum programming languages. It also includes implementations of major quantum algorithms: Shor’s Algorithm [53], the Quantum Fourier Transform [8], Bernstein-Vazirani [4], QAOA [10], and Grover’s Algorithm [18]. These circuits range in size from a few qubits and gates to hundreds of qubits and tens of thousands of gates (summarized in Appendix E [38]).

Global Experimental Setup. Unless otherwise noted, the following experimental conditions apply to all empirical evaluations. All compiler runs are allotted a 1-hour timeout on 16 cores of an AMD EPYC™ 7763 2.45 GHz Processor and 32GB of RAM accessed via a distributed research cluster. We choose this fixed timeout to compare Amaro, which continuously produces solutions over the course of a search, to a wide range of algorithms with different runtime characteristics. For example, the heuristic NISQMR compiler QISKIT can solve any instance within a minute, while the TIQMR tool SHAPER fails to terminate within the hour for most benchmarks. The parameters of Amaro are set to fixed default values for all case studies (see Appendix B [38]).

7.1 Noisy-Intermediate Scale Quantum (NISQMR)

Experimental setup. In the empirical evaluation for the NISQ problem, we target three different connectivity graphs from real QPUs: Rigetti Aspen-M [49], Google Sycamore [13], and IBM Eagle [22]. We compare against two state-of-the-art compilers. The first, QISKIT [26], is an industrial quantum programming toolkit which applies the SABRE algorithm [35]. Like Amaro, it greedily builds a solution by adding SWAP gates according to a heuristic scoring function. (We note that QISKIT has been in development for close to a decade and is heavily optimized for the NISQMR problem.) The other tool, QSYNTH2 [52], attempts to find globally optimal solutions. It encodes the QMR problem into a satisfiability problem and makes queries to an external SAT solver.

Results. The results are shown in Fig. 9. In the left and middle plots, we compare the solution quality of our approach to the baselines on the IBM Eagle architecture (results on other architectures

³Alternatively, the Amaro definition can be written directly in Rust. For cases where defining problem components requires complex computation, this may be preferable.

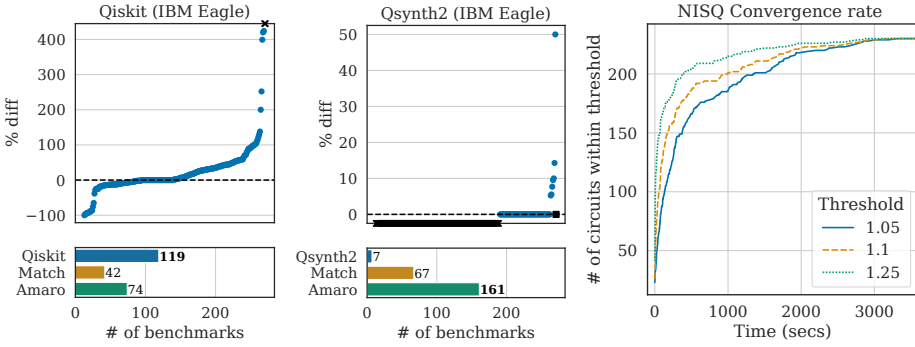


Fig. 9. NISQMR results

are similar and included in Appendix E [38]). Each point in the scatter plots represents a circuit, and the circuits are sorted by *percent difference* in cost. The percent difference is the quantity

$$\frac{\text{Amaro cost} - \text{baseline cost}}{\text{baseline cost}}$$

In this case, the cost is simply the number of added SWAP gates. Points below the dashed line at $y = 0$ thus represent circuits where our approach produces a better solution. Benchmarks where the baseline compiler, Amaro, or both fail to find a solution are indicated by black marks on the bottom of the plot (baseline timeout), on the dashed line (both timeout), and on the top of the plot (Amaro timeout). The bar plots shown under the scatter plots aggregate the number of benchmarks below the dashed line of equivalent performance (labeled “Amaro”), on the line (labeled “Match”), and above the line (labeled with the baseline), including timeouts.

Overall, Amaro is competitive with the specialized tools. For example, in the left plot we see that Amaro matches or outperforms QISKIT on roughly 48% of benchmarks, with QISKIT strictly outperforming Amaro on the remaining 52%, including one timeout.⁴ Though in the aggregate the two tools have similar performance, there are outliers in both directions where one significantly outperforms the other. The points near and above 400% difference all correspond to relatively wide and shallow circuits, with over 100 qubits and under 300 cx gates. These are likely workloads where specialized mapping algorithms can find a good initial qubit map among a large search space.

The comparison to QSYNTH2 in Fig. 9(middle). QSYNTH2 is optimal, so there are no cases where Amaro finds a solution strictly lower in cost. Amaro solves 161 benchmarks where QSYNTH2 does not terminate. Excluding timeouts, Amaro matches the optimal solution in 66/73 cases.

The line plot on the right of Fig. 9 shows the rate at which Amaro converges to a solution. We plot the number of circuits for which Amaro has found a solution which is close to the final cost of the best solution found within the 1hr time out. Each line represents a different threshold for “close.” The solid blue line is the strictest requirement—current solution is within 5% of the final solution cost—which is why it is the lowest line at any particular time point. In 80% of the benchmarks, a solution meeting the 10% threshold is found within 492 seconds.

7.2 NISQ Variable Error (NISQ-VE)

Experimental Setup. To evaluate performance on the NISQ-VE problem, we augmented the three connectivity graphs from NISQMR above with error rates for each coupling link between pairs of

⁴8 circuits with too many qubits to be executed on this architecture are excluded from the total

qubits. These error rates were generated by sampling uniformly from the range $[10^{-3}, 10^{-1}]$, to match the scale of two-qubit errors observed on the actual hardware [15, 22, 49].

Results. The QISKIT compiler can be configured to solve NISQ-VE. We compare the success probability of Amaro to QISKIT in Fig. 10(left). Here, the y-axis represents the *success probability ratio* which is the QISKIT estimated success probability divided by the Amaro estimated success probability. Note the logarithmic scale. Points below the dashed line at $y = 1$ represent circuits where our approach produces a better solution. We exclude benchmarks where one of the computed success probabilities is extremely low, below 10^{-16} , for numerical stability. We find that Amaro and QISKIT are essentially equivalent on average for the variation-aware variant of the NISQ problem.

Even though QISKIT produces solutions with fewer SWAP gates (as seen in the results for the NISQ problem), this is counterbalanced in the variation-aware setting by the fact that Amaro chooses SWAP gates with lower error rates. These results suggest that Amaro is an appropriate choice for compilation to NISQ devices in cases where accurate error rates are readily available. Moreover, since there are cases where QISKIT significantly outperforms Amaro and vice versa, a portfolio compilation approach may be effective. The convergence rate of Amaro for NISQ-VE is in Fig. 10(right). Results are generally similar to NISQMR: the time to reach the 10% threshold for 80% of circuits is 600s.

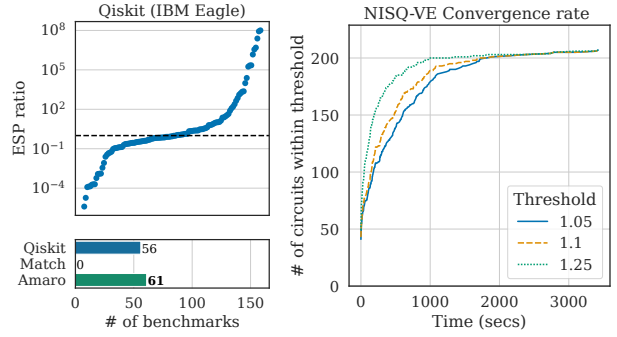


Fig. 10. NISQ-VE results

7.3 Surface Code Mapping and Routing (scmr)

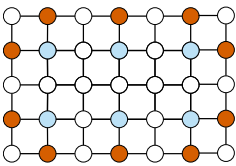


Fig. 11. 6-qubit Compact Architecture

Experimental Setup. For the scmr problem, we compare our approach against the state-of-the-art tool DASCOT [40]. We target the “Compact Architecture” used in the original evaluation of DASCOT. We choose the Compact Architecture because it represents a challenging chase for scmr with limited ancilla qubits available for routing. Logical qubits are arranged almost linearly, with a row of logical qubits, a central routing row, then another row of logical qubits. Magic state qubits are available along the perimeter of the QPU. An instance of the Compact Architecture with 6 qubits is shown in Fig. 11 (blue and orange vertices denote map locations and magic states respectively). All circuits are compiled for the smallest

possible Compact Architecture QPU matching the experimental setup originally used to evaluate DASCOT [40].

Results. Fig. 12(left) shows a comparison to DASCOT in terms of solution quality—number of states in a solution. Overall, Amaro outperforms DASCOT: DASCOT finds a better solution for 35% of cases, Amaro for 57%, and the solutions have the exact same number of cost or both tools timeout in the remaining 8%. Solutions are often close in quality when both tools terminate, within 6% for half of the benchmarks. The two circuits with percent difference over 100% are both large Grover’s algorithm circuits. Amaro struggles on this relatively large application compared to DASCOT. In Fig. 12(right), we plot the convergence rate for the scmr problem. Notice that compared to the

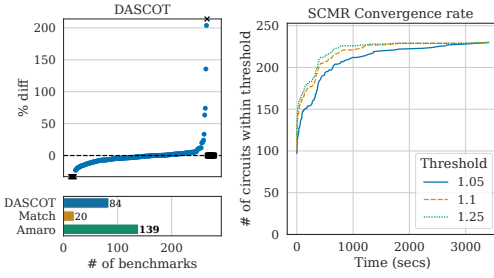


Fig. 12. SCMR results

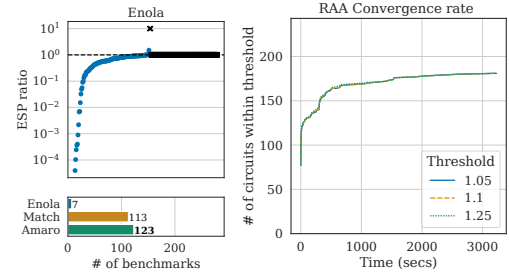


Fig. 13. RAA results

previous two problems, the first plotted point is higher on the y -axis. That is, it is more likely that early candidate solutions will be strong choices that are difficult to improve upon. The time to reach the 10% threshold on 80% of the circuits is slightly lower at 324 seconds.

7.4 Reconfigurable Atom Arrays (RAA)

Experimental setup. We compare against the state-of-the-art compiler ENOLA [57]. We use the same values for the empirically derived fidelity parameters (atom transfer fidelity, etc.) as in the original evaluation of ENOLA.

Results. Amaro is able to find slightly higher quality solutions in almost all cases. Fig. 13(left) is the same type of plot as Fig. 10(left). The y -axis is the success probability ratio—the ENOLA estimated success probability divided by the Amaro estimated success probability. Amaro produces a better solution in about 97% of cases where at least one tool terminates, with a median percent difference of 75%. In the convergence plot, Fig. 13(right), we see that the lines for the three thresholds are completely overlapping. The overlap suggests that all benchmarks have a clear best solution found by Amaro, and no alternative within 25% of its cost. The time to reach the 10% threshold on 80% of the circuits is 304 seconds.

7.5 Multi-qubit Lattice Surgery (MLSS)

Experimental setup. To evaluate on the multi-qubit lattice surgery problem, we converted all benchmark circuits to multi-body Pauli product rotation form [37]. The baseline approach presented in Silva et al. [54] is not available for reuse, so we compare against a theoretical lower-bound. Any solution for a circuit with depth d must have at least d states to respect the logical dependencies.

Results. Amaro is able to reach the theoretical lower-bound for most benchmarks within the timeout, though no solution is found for 13. Except for these hard instances, Amaro converges quickly for this problem, all best solutions (excluding timeouts) are found within the first 10 minutes or so of the search. The time to reach the 10% threshold on 80% of the circuits is 44 seconds.

7.6 Interleaved Logical Qubits (ILQ)

Experimental Setup. For this problem, we compared against the simple compilation workflow used to evaluate the viability of the ILQ architecture as compared to traditional surface code designs [60]. This baseline compiler applies a simple greedy algorithm which tries to group interacting qubits into the same stack, then routes instructions as soon as possible. We target the interleaved version of the Compact Architecture with a stack depth of 4, chosen based on the observation by Viszlai et al. [60] that this value is an inflection point, with diminishing returns for large stack sizes.

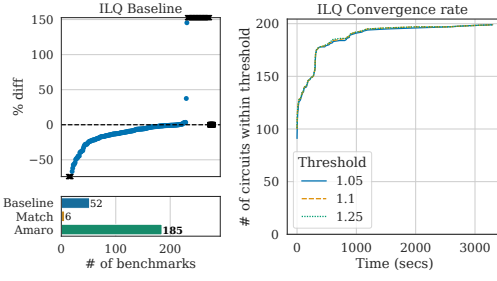


Fig. 14. ILQ results

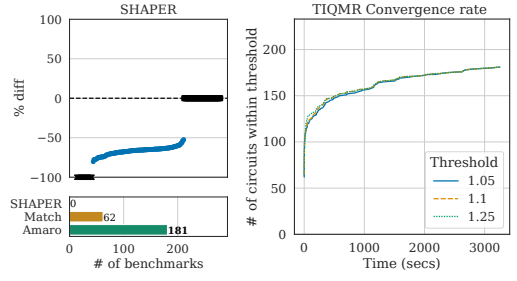


Fig. 15. TIQMR results

Results. Amaro outperforms the ILQ baseline on the majority of benchmarks as shown in Fig. 14(left). As a greedy algorithm, the baseline is able to solve more benchmarks. However, Amaro finds a better solution in virtually all cases where it terminates, such that 76% of circuits fall in the Amaro better category. The convergence data for the ILQ problem is shown in Fig. 14(right). Results are similar to RAA, with no distinction between the three thresholds and requiring 303 seconds to reach the 10% threshold on 80% of benchmarks.

7.7 Trapped-ions (TIQMR)

Experimental Setup. For the trapped-ion problem, we compared against the SHAPER algorithm [3]. We target the G2x3 architecture [29, 42], which consists of 6 traps arranged in two rows with three traps each. For each circuit, we assume the smallest trap size with capacity for all of the circuit qubits ($\lceil n/6 \rceil$ where n is the number of circuit qubits).

Results. As shown in Fig. 15(left), Amaro significantly outperforms SHAPER, always reducing the cost by at least 50% when both tools terminate. It also finds solutions to 27 benchmarks where SHAPER does not. The TIQMR problem is non-interfering, so here the search over maximal states is not the source of the benefits. Instead, the results are sensitive to the choice of initial qubit map, and Amaro searches over this space more comprehensively than SHAPER. The convergence results for TIQMR, shown in Fig. 15(right) are similar to other problems, with a similar time of 352 seconds to reach the 10% threshold on 80% of circuits. However, we note a less pronounced plateau at the end of the timeout. This suggests a longer tail of hard instances for this problem.

7.8 Ablation Study

To address Q3, we isolate the effect of the two search subroutines of Amaro with an ablation study, with the results shown in Fig. 16.

We evaluate the initial qubit map search by comparing to a version of Amaro that chooses a single initial qubit map, rather than applying simulated annealing search to find the best initial qubit map. Fig. 16(left) shows the percent increase in cost from this ablation for each case study, averaged across the entire benchmark suite (for detailed results, see Appendix E [38]). Here we see some improvement in quality attributable to the initial map search in all but two case studies (on which it has essentially no effect), including over 25% improvement in both variants of the NISQ problem.

We also assess the gains from searching for the best maximal state. To this end, we evaluate against a version of Amaro which iterates over a layer in a random order, rather than searching for the best order. The results are in the right plot; case studies with non-interferences are excluded

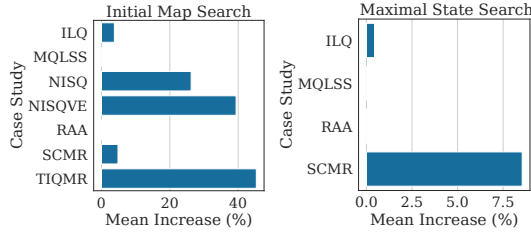


Fig. 16. Ablation study

because maximal state search is disabled. In this case, we see a more modest effect, with the SCMR case benefiting the most from maximal state search, at about an 8% average difference.

8 Related Work

Qubit mapping and routing. A large body of prior work has extensively explored QMR as a manually engineered compiler pass [67], and we have discussed representative QMR algorithms throughout this paper (see Table 1). Our solver shares some structure with the SABRE algorithm for NISQMR [35] which also constructs a single time step at a time, choosing the best transition at each stage. We also borrow algorithmic insights from DASCOT [40], which applies simulated annealing to SCMR. However, DASCOT uses a two-phase search where the initial map is scored relative to a heuristic function, as opposed to our joint optimization which explicitly computes a solution for each candidate. Existing algorithms are typically specialized to the constraints of a particular QMR problem; to our knowledge, this work is the first to automate synthesis of the QMR compiler pass.

Compiler Synthesis. We are inspired by efforts to automatically synthesize compilers in other domains. A classical example is parser generators [28, 34, 47], which automatically produce a parser from a grammar. Another line of work in this vein is automated program optimization via rewrite-rule synthesis. Rather than relying on hand-crafted optimizations, rewrite rule synthesizers automatically generate sound substitutions. Rewrite rule synthesis has been applied to many domains [27, 43, 45]. Within quantum computing, rewrite-rule synthesizers have also been developed for quantum-circuit optimization [64, 65], a compiler pass that is typically distinct from QMR.

Combinatorial search in compilation. QMR is a compilation pass that requires a combinatorial search to find the best solution satisfying constraints of the target hardware. In this way it is related to other compiler problems like superoptimization [51] and FPGA/VLSI routing [33]. QMR even relies on some of the same subroutines as VLSI routing including path-finding and Steiner tree construction [32, 58].

System-modeling DSLs. Finally, our use of a domain-specific language for describing the constraints of quantum hardware builds on a history of domain-specific languages for modeling systems. Examples from software verification include the Spin/Promela model checking framework [50] for concurrent systems and the Alloy Analyzer [23]. In electronic design analysis, hardware description languages like Verilog [2] and VHDL [1] describe the structure of classical hardware just as Amaro describes the structure of quantum hardware.

9 Conclusion

There are numerous parallel attempts at building quantum computers using a dizzying array of qubit hardware, physical layouts, and error-correction schemes. Each combination requires a carefully constructed compiler to map quantum programs onto the quantum processor while satisfying

its idiosyncratic constraints. We have presented an approach for automatically constructing a mapping and routing compiler for a given quantum processor. We started from the observation that all mapping and routing problems share a similar structure, which we can define using a simple domain-specific language. Using a generic solving algorithm, we demonstrated that we can construct powerful mapping and routing compilers for a wide range of quantum processors. We see two avenues for future research: (1) Improving the runtime and accuracy of the search algorithm, perhaps using reinforcement learning to construct mapping and routing policies that can transfer between QPUs. (2) Combining mapping and routing with circuit-optimization synthesis [64, 65]. This allows us to generate compilers that co-optimize the circuit and its mapping onto the device.

Acknowledgments

We thank the anonymous reviewers for their insightful feedback and suggestions. Thanks to Robert Rand, Moin Qureshi, and members of the madPL group for their feedback on early versions of this work. We are grateful to the CHTC team for maintaining the computing resources which enabled our empirical evaluation [7]. This work is supported by NSF grants #2340267 and #2212232 and awards from Meta and Amazon.

References

- [1] 2019. IEEE Standard for VHDL Language Reference Manual. *IEEE Std 1076-2019* (2019), 1–673. doi:10.1109/IEEESTD.2019.8938196
- [2] 2024. IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language. *IEEE Std 1800-2023 (Revision of IEEE Std 1800-2017)* (2024), 1–1354. doi:10.1109/IEEESTD.2024.10458102
- [3] Bao Bach, Ilya Safro, and Ed Younis. 2025. Efficient Compilation for Shuttling Trapped-Ion Machines via the Position Graph Architectural Abstraction. arXiv:2501.12470 [quant-ph] <https://arxiv.org/abs/2501.12470>
- [4] Ethan Bernstein and Umesh Vazirani. 1997. Quantum Complexity Theory. *SIAM J. Comput.* 26, 5 (1997), 1411–1473. arXiv:<https://doi.org/10.1137/S0097539796300921> doi:10.1137/S0097539796300921
- [5] Dolev Bluvstein, Simon J. Evered, Alexandra A. Geim, Sophie H. Li, Hengyun Zhou, Tom Manovitz, Sepehr Ebadi, Madelyn Cain, Marcin Kalinowski, Dominik Hangleiter, J. Pablo Bonilla Ataides, Nishad Maskara, Iris Cong, Xun Gao, Pedro Sales Rodriguez, Thomas Karolyshyn, Giulia Semeghini, Michael J. Gullans, Markus Greiner, Vladan Vuletić, and Mikhail D. Lukin. 2023. Logical quantum processor based on reconfigurable atom arrays. *Nature* 626, 7997 (Dec. 2023), 58–65. doi:10.1038/s41586-023-06927-3
- [6] Sergey Bravyi and Alexei Kitaev. 2005. Universal quantum computation with ideal Clifford gates and noisy ancillas. *Physical Review A* 71, 2 (Feb. 2005). doi:10.1103/physreva.71.022316
- [7] Center for High Throughput Computing. 2006. Center for High Throughput Computing. doi:10.21231/GNT1-HW21
- [8] Don Coppersmith. 2002. An approximate Fourier transform useful in quantum factoring. *arXiv preprint quant-ph/0201067* (2002). doi:10.48550/arXiv.quant-ph/0201067
- [9] Alexander Cowtan, Silas Dikes, Ross Duncan, Alexandre Krajenbrink, Will Simmons, and Seyon Sivarajah. 2019. On the Qubit Routing Problem. *Leibniz Int. Proc. Inf.* 135 (2019), 5:1–5:32. arXiv:1902.08091 [quant-ph] doi:10.4230/LIPIcs.TQC.2019.5
- [10] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A Quantum Approximate Optimization Algorithm. arXiv:1411.4028 doi:10.48550/arXiv.1411.4028
- [11] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. 2012. Surface codes: Towards practical large-scale quantum computation. *Physical Review A* 86, 3 (sep 2012). doi:10.1103/physreva.86.032324
- [12] Google. 2024. *Google Quantum AI Roadmap*. Google. <https://quantumai.google/roadmap> Accessed: June 18, 2025.
- [13] Google Quantum AI. 2021. Sycamore Spec Sheet. <https://quantumai.google/hardware/datasheet/weber.pdf>
- [14] Google Quantum AI. 2023. Suppressing quantum errors by scaling a surface code logical qubit. *Nature* 614, 7949 (Feb. 2023), 676–681. doi:10.1038/s41586-022-05434-1
- [15] Google Quantum AI. 2024. Willow Spec Sheet. <https://quantumai.google/static/site-assets/downloads/willow-spec-sheet.pdf>
- [16] Google Quantum AI and Collaborators. 2024. Quantum error correction below the surface code threshold. *Nature* (Dec. 2024). doi:10.1038/s41586-024-08449-y
- [17] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. 2013. Quipper: a scalable quantum programming language. *SIGPLAN Not.* 48, 6 (June 2013), 333–342. doi:10.1145/2499370.2462177

- [18] Lov K. Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing* (Philadelphia, Pennsylvania, USA) (STOC '96). Association for Computing Machinery, New York, NY, USA, 212–219. doi:10.1145/237814.237866
- [19] Clare Horsman, Austin G Fowler, Simon Devitt, and Rodney Van Meter. 2012. Surface code quantum computing by lattice surgery. *New Journal of Physics* 14, 12 (dec 2012), 123011. doi:10.1088/1367-2630/14/12/123011
- [20] Fei Hua, Yanhao Chen, Yuwei Jin, Chi Zhang, Ari Hayes, Youtao Zhang, and Eddy Z. Zhang. 2021. AutoBraid: A Framework for Enabling Efficient Surface Code Communication in Quantum Computing. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) (MICRO '21). Association for Computing Machinery, New York, NY, USA, 925–936. doi:10.1145/3466752.3480072
- [21] IBM. 2025. *IBM Quantum Roadmap*. IBM. <https://www.ibm.com/roadmaps/quantum/> Accessed: June 18, 2025.
- [22] IBM Quantum. 2025. Quantum processing units. <https://quantum.ibm.com/services/resources>
- [23] Daniel Jackson. 2019. Alloy: a language and tool for exploring software designs. *Commun. ACM* 62, 9 (Aug. 2019), 66–76. doi:10.1145/3338843
- [24] Ali Javadi-Abhari, Pranav Gokhale, Adam Holmes, Diana Franklin, Kenneth R. Brown, Margaret Martonosi, and Frederic T. Chong. 2017. Optimized Surface Code Communication in Superconducting Quantum Computers. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture* (Cambridge, Massachusetts) (MICRO-50 '17). Association for Computing Machinery, New York, NY, USA, 692–705. doi:10.1145/3123939.3123949
- [25] Ali Javadi-Abhari, Shruti Patil, Daniel Kudrow, Jeff Heckey, Alexey Lvov, Frederic T. Chong, and Margaret Martonosi. 2014. Scaffold: A Framework for Compilation and Analysis of Quantum Computing Programs. In *Proceedings of the 11th ACM Conference on Computing Frontiers* (Cagliari, Italy) (CF '14). Association for Computing Machinery, New York, NY, USA, Article 1, 10 pages. doi:10.1145/2597917.2597939
- [26] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. 2024. Quantum computing with Qiskit. arXiv:2405.08810 [quant-ph] doi:10.48550/arXiv.2405.08810
- [27] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (Huntsville, Ontario, Canada) (SOSP '19). Association for Computing Machinery, New York, NY, USA, 47–62. doi:10.1145/3341301.3359630
- [28] S.C. Johnson. 1975. *YACC: Yet Another Compiler-Compiler*. Technical Report Comp. Sci. Tech. Rep. 32. Bell Laboratories.
- [29] D Kielpinski, C Monroe, and David Wineland. 2002. Architecture for a Large-Scale Ion-Trap Quantum Computer. 417 (2002-01-01 2002). doi:10.1038/nature00784
- [30] Youngseok Kim, Andrew Eddins, Sajant Anand, Ken Xuan Wei, Ewout Van Den Berg, Sami Rosenblatt, Hasan Nayfeh, Yantao Wu, Michael Zaletel, Kristan Temme, et al. 2023. Evidence for the utility of quantum computing before fault tolerance. *Nature* 618, 7965 (2023), 500–505.
- [31] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. 1983. Optimization by Simulated Annealing. *Science* 220, 4598 (1983), 671–680. doi:10.1126/science.220.4598.671
- [32] M.R Kramer and J van Leeuwen. 1984. The complexity of wire-routing and finding minimum area layouts for arbitrary vlsi circuits. *Advances in Computing Research* 2 (1984), 020342.
- [33] Ian Kuon, Russell Tessier, and Jonathan Rose. 2008. *FPGA Architecture: Survey and Challenges*. doi:10.1561/1000000005
- [34] M.E. Lesk. 1975. *LEX – A Lexical Analyzer Generator*. Technical Report Comp. Sci. Tech. Rep. 39. Bell Laboratories.
- [35] Gushu Li, Yufei Ding, and Yuan Xie. 2019. Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLOS '19). Association for Computing Machinery, New York, NY, USA, 1001–1014. doi:10.1145/3297858.3304023
- [36] Wan-Hsuan Lin, Jason Kimko, Bochen Tan, Nikolaj Bjørner, and Jason Cong. 2023. Scalable Optimal Layout Synthesis for NISQ Quantum Processors. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. doi:10.1109/DAC56929.2023.10247760
- [37] Daniel Litinski. 2019. A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery. *Quantum* 3 (March 2019), 128. doi:10.22331/q-2019-03-05-128
- [38] Abtin Molavi, Amanda Xu, Ethan Cecchetti, Swamit Tannu, and Aws Albarghouthi. 2025. Generating Compilers for Qubit Mapping and Routing. arXiv:2508.10781 [cs.PL] <https://arxiv.org/abs/2508.10781>
- [39] Abtin Molavi, Amanda Xu, Martin Diges, Lauren Pick, Swamit S. Tannu, and Aws Albarghouthi. 2022. Qubit Mapping and Routing via MaxSAT. *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)* (2022), 1078–1091. doi:10.1109/MICRO56248.2022.00077
- [40] Abtin Molavi, Amanda Xu, Swamit Tannu, and Aws Albarghouthi. 2025. Dependency-Aware Compilation for Surface Code Quantum Architectures. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 82 (April 2025), 28 pages. doi:10.1145/3720416

- [41] Prakash Murali, Jonathan M. Baker, Ali Javadi-Abhari, Frederic T. Chong, and Margaret Martonosi. 2019. Noise-Adaptive Compiler Mappings for Noisy Intermediate-Scale Quantum Computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLOS '19). Association for Computing Machinery, New York, NY, USA, 1015–1029. doi:10.1145/3297858.3304075
- [42] Prakash Murali, Dripto M. Debroy, Kenneth R. Brown, and Margaret Martonosi. 2020. Architecting Noisy Intermediate-Scale Trapped Ion Quantum Computers. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 529–542. doi:10.1109/ISCA45697.2020.00051
- [43] Julie L. Newcomb, Andrew Adams, Steven Johnson, Rastislav Bodik, and Shoaib Kamil. 2020. Verifying and improving Halide's term rewriting system with program synthesis. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 166 (Nov. 2020), 28 pages. doi:10.1145/3428234
- [44] Michael A. Nielsen and Isaac L. Chuang. 2011. *Quantum Computation and Quantum Information: 10th Anniversary Edition* (10th ed.). Cambridge University Press, USA.
- [45] Pavel Pancheckha, Alex Sanchez-Stern, James R. Wilcox, and Zachary Tatlock. 2015. Automatically improving accuracy for floating point expressions. *SIGPLAN Not.* 50, 6 (June 2015), 1–11. doi:10.1145/2813885.2737959
- [46] Anouk Paradis, Jasper Dekoninck, Benjamin Bichsel, and Martin Vechev. 2024. Synthetiq: Fast and Versatile Quantum Circuit Synthesis. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 96 (April 2024), 28 pages. doi:10.1145/3649813
- [47] Terence Parr. 2013. *The Definitive ANTLR 4 Reference* (2nd ed.). Pragmatic Bookshelf.
- [48] J. M. Pino, J. M. Dreiling, C. Figgatt, J. P. Gaebler, S. A. Moses, M. S. Allman, C. H. Baldwin, M. Foss-Feig, D. Hayes, K. Mayer, C. Ryan-Anderson, and B. Neyenhuis. 2021. Demonstration of the trapped-ion quantum CCD computer architecture. *Nature* 592, 7853 (2021). doi:10.1038/s41586-021-03318-4
- [49] Rigetti Computing. 2025. Rigetti Systems. <https://qcs.rigetti.com/qpus>
- [50] Rob Gerth. 1997. Concise Promela reference. <https://spinroot.com/spin/Man/Quick.html>
- [51] Eric Schkufza, Rahul Sharma, and Alex Aiken. 2013. Stochastic superoptimization. *ACM SIGARCH Computer Architecture News* 41, 1 (2013), 305–316.
- [52] Irfansha Shaik and Jaco van de Pol. 2024. Optimal Layout Synthesis for Deep Quantum Circuits on NISQ Processors with 100+ Qubits. In *27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 305)*, Supratik Chakraborty and Jie-Hong Roland Jiang (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 26:1–26:18. doi:10.4230/LIPIcs.SAT.2024.26
- [53] P.W. Shor. 1994. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*. 124–134. doi:10.1109/SFCS.1994.365700
- [54] Allyson Silva, Xiangyi Zhang, Zak Webb, Mia Kramer, Chan-Woo Yang, Xiao Liu, Jessica Lemieux, Ka-Wai Chen, Artur Scherer, and Pooya Ronagh. 2024. Multi-qubit Lattice Surgery Scheduling. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.TQC.2024.1
- [55] Marcos Yukio Siraichi, Vinicius Fernandes dos Santos, Caroline Collange, and Fernando Magno Quintao Pereira. 2018. Qubit allocation. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization* (Vienna, Austria) (CGO '18). Association for Computing Machinery, New York, NY, USA, 113–125. doi:10.1145/3168822
- [56] Bochen Tan, Dolev Bluvstein, Mikhail Lukin, and Jason Cong. 2024. Compiling Quantum Circuits for Dynamically Field-Programmable Neutral Atoms Array Processors. *Quantum* 8 (03 2024), 1281. doi:10.22331/q-2024-03-14-1281
- [57] Daniel Bochen Tan, Wan-Hsuan Lin, and Jason Cong. 2025. *Compilation for Dynamically Field-Programmable Qubit Arrays with Efficient and Provably Near-Optimal Scheduling*. Association for Computing Machinery, New York, NY, USA, 921–929. <https://doi.org/10.1145/3658617.3697778>
- [58] Hao Tang, Gengeng Liu, Xiaohua Chen, and Naixue Xiong. 2020. A Survey on Steiner Tree Construction and Global Routing for VLSI Design. *IEEE Access* 8 (2020), 68593–68622. doi:10.1109/ACCESS.2020.2986138
- [59] Swamit S. Tannu and Moinuddin K. Qureshi. 2019. Not All Qubits Are Created Equal: A Case for Variability-Aware Policies for NISQ-Era Quantum Computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLOS '19). Association for Computing Machinery, New York, NY, USA, 987–999. doi:10.1145/3297858.3304007
- [60] Joshua Vizslai, Sophia Lin, Siddharth Dangwal, Conor Bradley, Vikram Ramesh, Jonathan Baker, Hannes Bernien, and Frederic T. Chong. 2025. Interleaved Logical Qubits in Atom Arrays. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 261–274. doi:10.1109/HPCA61900.2025.00030
- [61] Hanrui Wang, Pengyu Liu, Daniel Bochen Tan, Yilian Liu, Jiaqi Gu, David Z. Pan, Jason Cong, Umut A. Acar, and Song Han. 2024. Atomique: A Quantum Compiler for Reconfigurable Neutral Atom Arrays. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 293–309. doi:10.1109/ISCA59077.2024.00030
- [62] Robert Wille and Lukas Burgholz. 2023. MQT QMAP: Efficient Quantum Circuit Mapping. In *Proceedings of the 2023 International Symposium on Physical Design (ISPD '23)*. ACM, 198–204. doi:10.1145/3569052.3578928

- [63] Robert Wille, Daniel Große, Lisa Teuber, Gerhard W. Dueck, and Rolf Drechsler. 2008. RevLib: An Online Resource for Reversible Functions and Reversible Circuits. In *38th International Symposium on Multiple Valued Logic (ismvl 2008)*. 220–225. doi:10.1109/ISMVL.2008.43
- [64] Amanda Xu, Abtin Molavi, Lauren Pick, Swamit Tannu, and Aws Albarghouthi. 2023. Synthesizing quantum-circuit optimizers. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 835–859. doi:10.1145/3591254
- [65] Mingkuan Xu, Zikun Li, Oded Padon, Sina Lin, Jessica Pointing, Auguste Hirth, Henry Ma, Jens Palsberg, Alex Aiken, Umut A. Acar, and Zhihao Jia. 2022. Quartz: superoptimization of Quantum circuits. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (San Diego, CA, USA) (PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, 625–640. doi:10.1145/3519939.3523433
- [66] Jiong Yang, Yaroslav A. Kharkov, Yunong Shi, Marijn J. H. Heule, and Bruno Dutertre. 2024. Quantum Circuit Mapping Based on Incremental and Parallel SAT Solving. In *27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 305)*, Supratik Chakraborty and Jie-Hong Roland Jiang (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 29:1–29:18. doi:10.4230/LIPIcs.SAT.2024.29
- [67] Chenghong Zhu, Xian Wu, Zhaohui Yang, Jingbo Wang, Anbang Wu, Shenggen Zheng, and Xin Wang. 2025. Quantum Compiler Design for Qubit Mapping and Routing: A Cross-Architectural Survey of Superconducting, Trapped-Ion, and Neutral Atom Systems. *arXiv preprint arXiv:2505.16891* (2025).
- [68] Alwin Zulehner, Alexandru Paler, and Robert Wille. 2018. An efficient methodology for mapping quantum circuits to the IBM QX architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 7, 1226–1236. doi:10.1109/TCAD.2018.2846658

Received 2025-07-10; accepted 2025-11-06